



Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Самарский государственный технический университет»
(ФГБОУ ВО «СамГТУ»)

Кафедра «Высшая математика»

AI–агенты

название работы

Учебная практика

Выполнил студент
__1__ курса, __113М__ группы
Бекиш Егор Павлович

подпись

Руководитель
Доцент, к. ф.-м. н.
Гурская Альбина Валентиновна

подпись

Работа сдана
«__» _____ 202__ г.

Оценка _____

Самара, 202__ г.

Оглавление

Введение	3
1. ОПИСАНИЕ ЯЗЫКОВОЙ МОДЕЛИ	4
1.1 Проблематика	4
1.2 Развитие больших языковых моделей.....	4
1.2.1 Появление обработки естественного языка	4
1.2.2 Развитие векторного представления слов	5
1.2.3 Виды языковых моделей	9
1.2.4 Выборка необходимых документов и данных.....	18
1.3 Развитие AI–агентов.....	21
2. РЕАЛИЗАЦИЯ.....	23
2.1 Получение необходимых параметров модели	23
Заключение	28
Литературы	29

Введение

Языковые модели [1] являются важной и активно развивающейся областью, которая за последние несколько лет привлекает большое внимание промышленников, научных и академических кругов.

В современном мире объемы информации с каждым днем растут все больше и больше, поэтому анализировать информацию становится труднее, как и выбрать более корректную. В организациях существуют собственные корпоративные документы, которые имеют тоже не маленький объем. Поэтому, чтобы повысить собственное качество и работоспособность им бы хотелось избегать анализа лишнего материала, т.к. на нее тратится большое время.

AI-агент – система на основе языковых моделей, способна не просто генерировать ответ, но и самостоятельно планировать последовательность действий, взаимодействовать с внешними инструментами и достигать заданной цели с качественным контролем человека. В отличие от классических языковых моделей, работающих в режиме «запрос – ответ», AI-агент может разбить сложную задачу на этапы: уточнить исходные данные, запросить недостающую информацию из корпоративных документов, отфильтровать нерелевантные источники и выполнить целевое действие.

1. ОПИСАНИЕ ЯЗЫКОВОЙ МОДЕЛИ

1.1 Проблематика

За последние несколько десятилетий произошел значительный рост объемов информации, что потребовало тщательного анализа поступающих данных, существенно замедляя как образовательный, так и рабочий процесс. В этих условиях традиционные методы работы с информацией становятся неэффективными и требуют значительных ресурсов для поддержания актуальности и доступности информации. Языковые модели (LLM), такие как ChatGPT [2], предлагают инновационные решения для автоматизации обработки и анализа данных, улучшения поиска и доступа к знаниям для пользователей и сотрудников компаний, а также поддержки принятия решений. Однако, несмотря на очевидные преимущества, многие пользователи и компании еще не полностью осознали потенциал LLM и сталкиваются с вызовами при их внедрении.

1.2 Развитие больших языковых моделей

1.2.1 Появление обработки естественного языка

В 1950-м году Алан Тьюринг опубликовал статью «Computing Machinery and Intelligence» [3], в которой рассматривал вопрос: «Могут ли машины мыслить?», где, для начала, нужно определить термины в данном вопросе – «мыслить» и «машина». В дальнейшем выдвинул идею, что человек может взаимодействовать с машиной на естественном языке. Спустя 16 лет была опубликована первая программа по генерации текста ELIZA, которая имитировала психотерапевта, используя шаблоны и правила.

В 70 – 80-х годах развивался метод машинного обучения на базе правил, где данные правила задавались вручную, но системы на тот момент не поддерживали масштабируемость.

1.2.2 Развитие векторного представления слов

Векторное представление слов – вектор в пространстве с фиксированной размерностью. На вход подаются коллекция документов, на выходе получаем векторное представление из коллекции документов.

One-hot encoding [4] – метод представления в векторном виде. Создается словарь фиксированного размера N , где для каждого слова из словаря соответствует вектор размера N , каждый вектор состоит из одной единицы на i -м месте и $(N - 1)$ нулей, где i – номер слова в словаре. Таким образом, запись данного метода представлена на рисунке 1.1.

Например, $n=50.000$ слов.

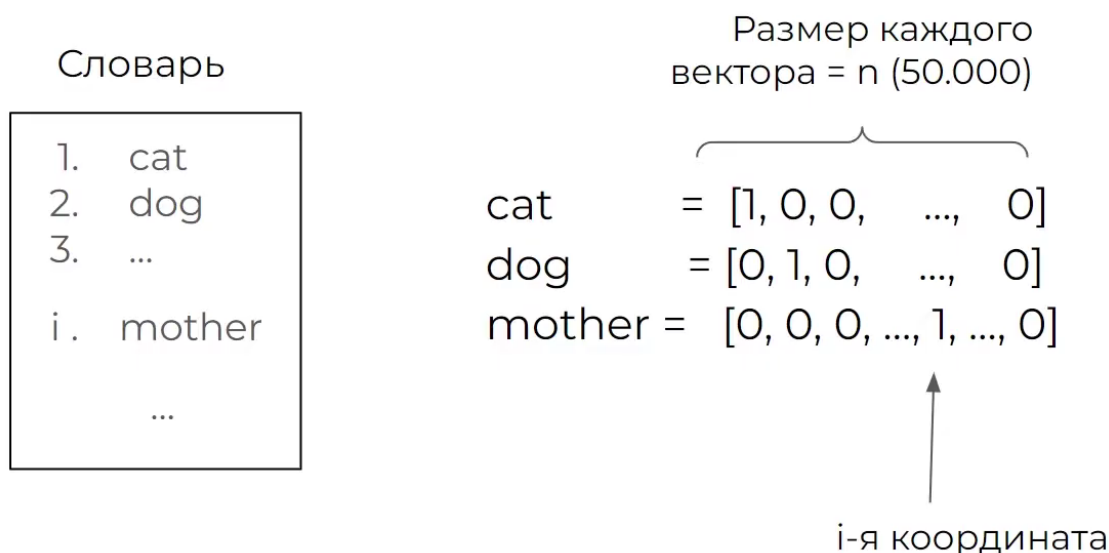


Рисунок 1.1 – One-hot encoding

Перейдем к методу, который учитывает данную важность. TF-IDF [5] – вычисляет важность каждого слова в документе относительно количества его употреблений в данном документе и во всей коллекции текстов. Этот метод позволяет выделить ключевые слова и понять, какие слова имеют больший вес для определенного документа в контексте всей коллекции. Или же более простыми словами – мера важности слова t для документа d среди документов D .

Алгебраическая форма записи выглядит следующим образом:

$$tf - idf(t, d, D) = tf(t, d) \times idf(t, D) \quad (1.1)$$

где t – входное слово;

d – текущий документ;

D – коллекция документов;

$tf(t, d)$ – относительная частота встречаемого слова в документе;

$idf(t, D)$ – обратная частота встречаемого слова в наборе документов.

Относительная частота рассчитывается следующим образом:

$$tf(t, d) = \frac{n_t}{\sum_k n_k} \quad (1.2)$$

где t – входное слово;

d – документ, в котором ищется слово t ;

n_t – количество слова t в документе d ;

$\sum_k n_k$ – количество слов в документе d .

Обратная частота рассчитывается следующим образом:

$$idf(t, D) = \log \frac{|D|}{|\{d_i \in D | t \in d_i\}|} \quad (1.3)$$

где t – входное слово;

D – коллекция документов, их количество;

$|\{d_i \in D | t \in d_i\}|$ – количество документов, где слово t встречается.

Следовательно, значение обратной частоты показывает, чем оно больше, т.е. количество слов t встречается реже во всех документах, тем оно является более важным по смыслу в текущем документе, в противном случае, как было сказано про артикли и предлоги до этого, их количество велико, что говорит нам о том, что их важность очень мала и не несет никакой смысловой нагрузки. Теперь, когда можно найти важность каждого слова среди документов, строятся векторы документов из данных значений, который имеет размер словаря N и на i -й позиции вектора будет стоять значение i -го слова данного документа. Для наглядности пример можно увидеть на рисунках 1.2 – 1.3.

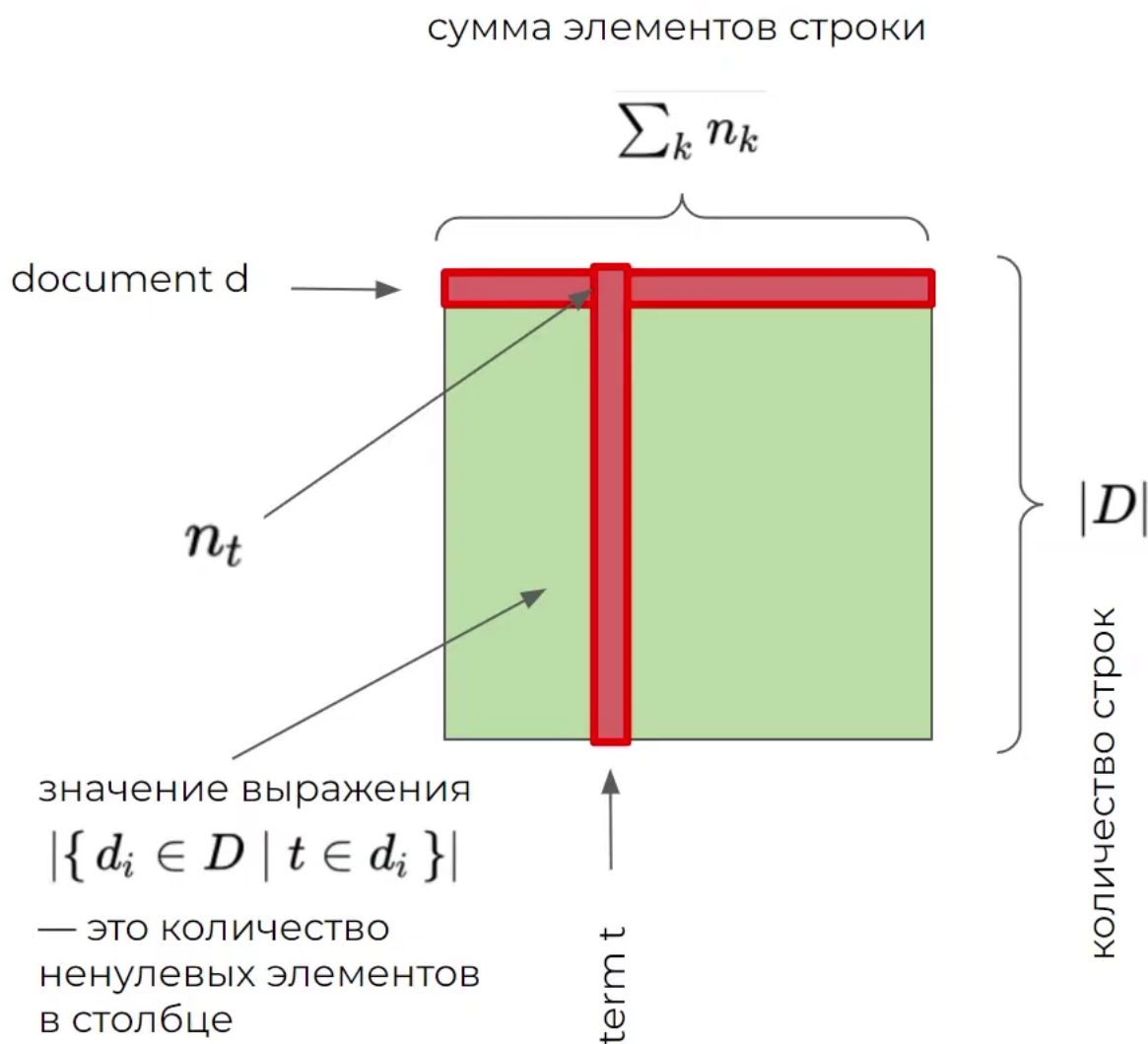


Рисунок 1.2 – Матрица TF-IDF

1. **a dog eats meat**
2. **a dog hunts cat**

TF-IDF векторы для этих предложений:

- | | | | | | | | | | |
|----|---|--------|--------|------------|-------|------------|-------------|-------------|------|
| 1. | [| 0, ... | 0, ... | 0.075, ... | 0, | ..., | 0, ..., | 0, | ...] |
| 2. | [| 0, ... | 0, ... | 0, | ..., | 0.075, ... | 0.075, ..., | 0.075, ...] | |
| | | ↑ | ↑ | ↑ | ↑ | ↑ | ↑ | ↑ | |
| | | a | dog | eats | hunts | cat | meat | | |

Рисунок 1.3 – Векторное представление TF-IDF

На основе предыдущего метода была разработана модификация TF – IDF – BM25 [6] – полнотекстовый поиск заключается в нахождении *наиболее релевантных* запросу документов из множества вариантов.

Алгебраическая форма записи выглядит следующим образом:

$$score(D, Q) = \sum_{i=1}^n IDF(q_i) \cdot TF(q_i, D) \quad (1.4)$$

где D – документ;

Q – запрос;

$IDF(q_i)$ – обратная частота документа;

$TF(q_i, D)$ – частота слова в документе.

Часто слова в документе выглядит следующим образом:

$$TF(q_i, D) = \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1} \cdot \frac{1}{\left(1 - b + b \cdot \frac{|D|}{avgdl}\right)} \quad (1.5)$$

где q_i – запрос;

D – документ;

$f(q_i, D)$ – частота запроса в документе;

k_1, b – свободные коэффициенты ($const, k_1 \in [1.2; 2.0], b \in [0; 1]$);

$|D|$ – длина документа;

$avgdl$ – средняя длина документа в нашей коллекции.

Обратная частота документа выглядит следующим образом:

$$IDF(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} \right) \quad (1.6)$$

где N – общее количество элементов в коллекции;

$n(q_i)$ – количество документов, содержащих элемент запроса;

$N - n(q_i)$ – это количество документов, не содержащих элемент запроса.

Сам алгоритм BM25, как и было выше сказано основан на принципе вероятностного ранжирования – если извлечённые документы упорядочены по уменьшению вероятности релевантности данных, то эффективность системы оптимальна для этих данных.

1.2.3 Виды языковых моделей

С основными методами нахождения ранжированных слов, предложений и документов разобрались. Теперь необходимо как-то сформировать верное сформулированное и понятное по смыслу предложение. Одна из моделей – N-граммы – это последовательность из N подряд идущих символов, звуков, слов, слогов и т.п. в тексте [7]. Данная модель может быть:

1. Униграммой (1-граммы) – отдельные символы, слова и т.п.
2. Биграммой (2-граммы) – пара слов.
3. Триграммы (3-граммы) – тройка слов.

Статическая модель, предсказывающая или предугадывающая следующее слово, основанное на вероятностной модели. Рассчитывается по цепному правилу:

$$P(w_1^n) = P(w_1)P(w_2|w_1) \cdots P(w_n|w_1^{n-1}) = \prod_{k=1}^n P(w_k|w_1^{k-1}) \quad (1.7)$$

где w_n – слово в тексте;

$P(w_1^n)$ – общий случай N-граммы.

Как можем заметить, что вероятность слова w_n зависит от предыдущего w_1^{n-1} , а не от всего предыдущего текста, что было до этого, тогда общий случай выглядит следующим образом:

$$P(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_n)}{\text{count}(w_{i-1})} \quad (1.8)$$

Возникает проблема нулевых вероятностей, говорит о том, что если n-грамма не была в обучающихся данных, то вероятность равно нулю и модель не работает. Существует аддитивное сглаживание Лапласа, которое исправляет данную проблему. Алгебраическая запись выглядит следующим образом:

$$P(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + 1}{\text{count}(w_{i-1}) + V} \quad (1.9)$$

где V – размер словаря.

Модель трансформера [8] включает в себя два основных компонента: кодировщик (энкодер) и декодировщик (декодер), как показано на рисунке 1.4.

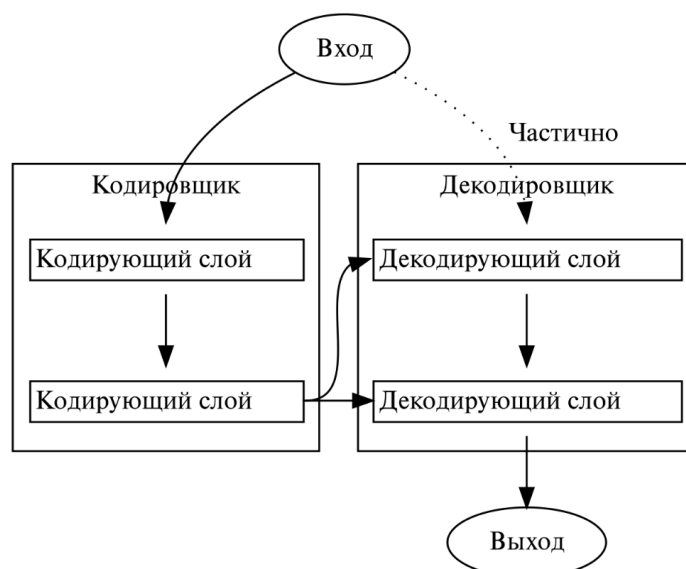


Рисунок 1.4 – Архитектура трансформера

На вход энкодера подается векторное представление последовательности, дополненное позиционными данными. В основе слоя декодера лежат методы и идеи, появившиеся в 2013 глубокого обучения, Word2Vec [9]. Они имеют другое название более распространенное – эмбединг [10]. Из набора последовательно берется пять слов, из них выбирается центральное и два слова до и после него – контекст центрального слова. Далее сдвигаемся на одно слово и повторяем данный алгоритм. Это можно увидеть на рисунках 1.5 – 1.6.

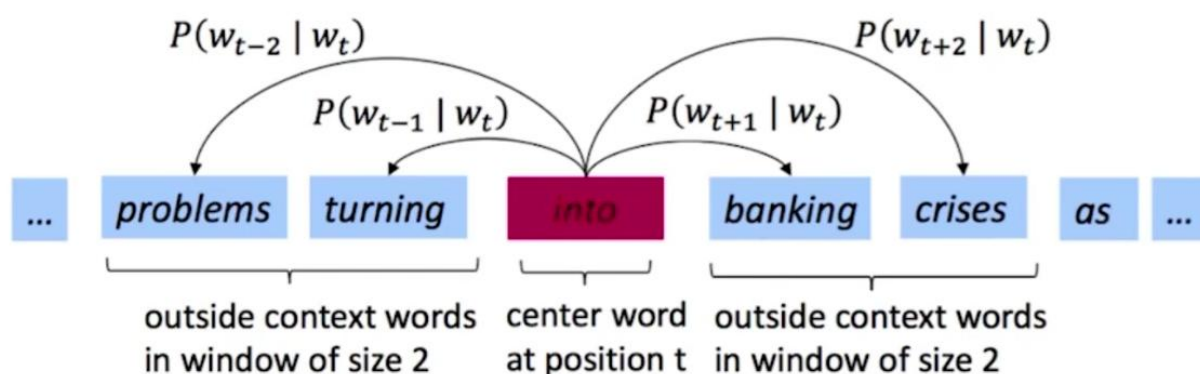


Рисунок 1.5 – Схематическое изображение алгоритма при центральном слове into

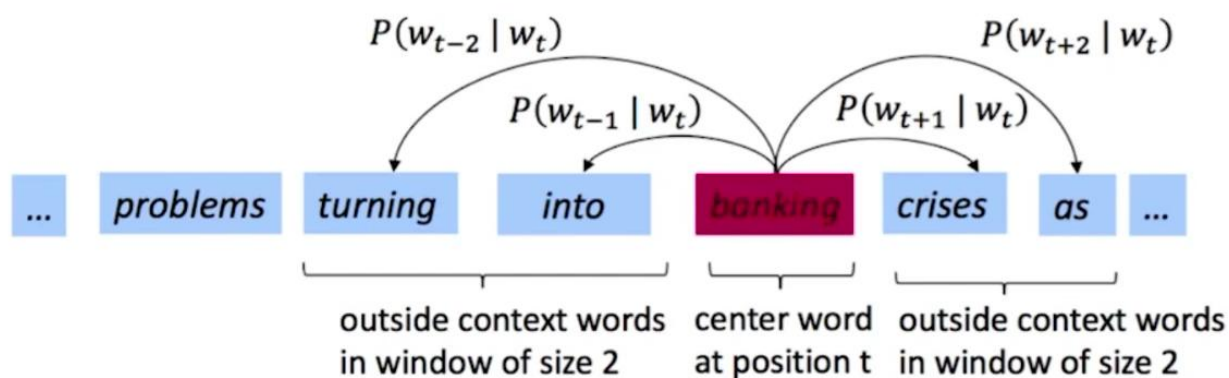


Рисунок 1.6 – Схематическое изображение алгоритма при центральном слове banking

Далее используется нейросеть, которая обучается на данных словах и позволяет сравнивать их смысл, контекст с помощью некоторой метрики, в основном применяется косинусное расстояние. Алгебраический вид, следующий:

$$similarity = \cos \theta = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}} \quad (1.10)$$

где A и B – векторы.

Тогда, такие обученные слова в векторном виде будут называться эмбедингом слов.

Так же необходимо учитывать контекст, поэтому воспользуемся алгоритмом выше, представленный на рисунках 1.5 – 1.6, когда у нас присутствует центральное слово и контекстом является два слова до и после него. На этих данных нейронная сеть будет учиться.

Вид данной нейросети будет показан на рисунке 1.7.

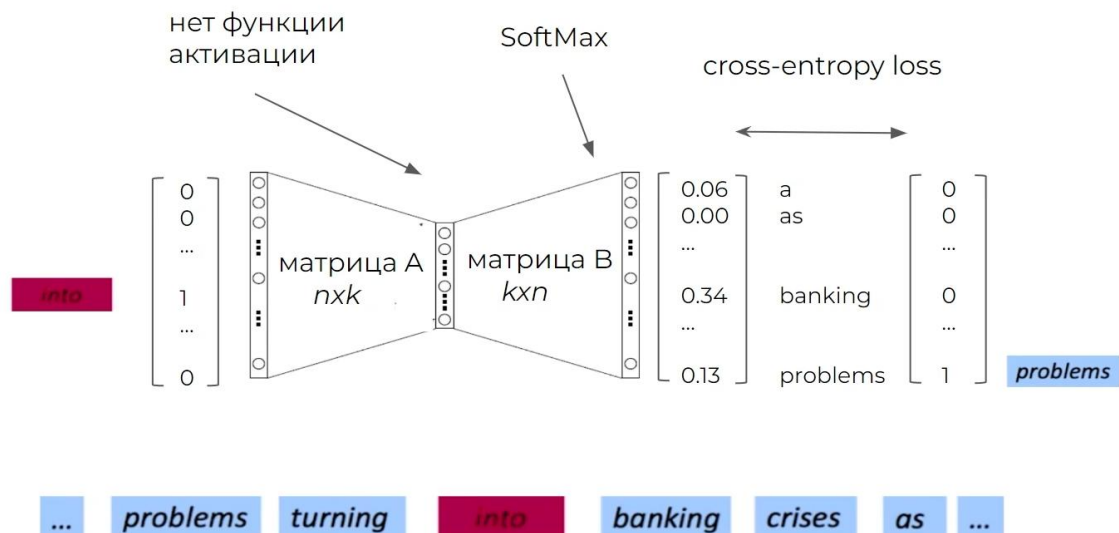


Рисунок 1.7 – Нейросеть модели Word2Vec

Как и было показано на рисунке 1.4 вход осуществляет через кодировщик. Сам энкодер имеет следующую схему на рисунке 1.8, где Word2Vec состоит в слое внимания.

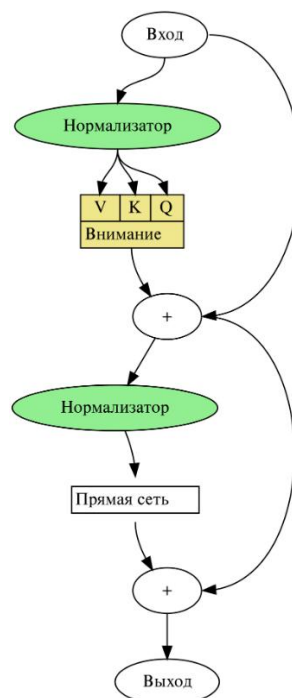


Рисунок 1.8 – Схема энкодера

Нормализатор. Применяется к входным данным перед слоем внимания. Стабилизирует обучение, уменьшая внутренний ковариационный сдвиг. Слой внимания вычисляет зависимость между всеми элементами последовательности, алгебраический вид, следующий:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.11)$$

где Q – запрос;

K – ключ;

V – значение;

d_k – размерность ключей.

Между каждым переходом происходит сложение или остаточное соединение, к выходу слоя внимания добавится исходный вход, т.е. нормализатор.

Прямая сеть FFN. Состоит из двух слоев с активацией, добавляя нелинейность и преобразует признаки.

В конце происходит выход слоя. После FFN применяется остаточное соединение и нормализатор. Данный результат отправляется на декодировщик. Схема декодировщика представлена на рисунке 1.9

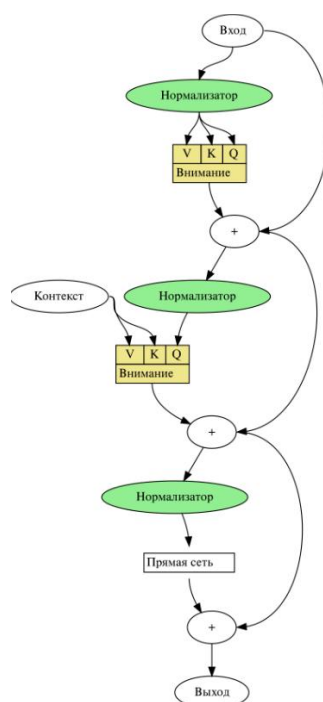


Рисунок 1.9 – Схема декодера

На вход декодера поступает результат, полученный на последнем шаге энкодера. Механизм нормализатора такой же, как и у декодера, поэтому опустим повторное описание. Оказавшись на слое самовнимания он будет являться маскировочным, и иметь следующий алгебраический вид:

$$MaskedAttention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (1.12)$$

где M – маска.

Маска используется для того, чтобы предотвратить утечку информации из будущих токенов в текущие.

Контекст от кодировщика. Выход последнего слоя кодировщика передаётся в декодировщик как ключи K и значения V для второго механизма внимания.

Остаточные этапы повторяются, как и в кодировщике. В итоге на выходе получаем вектор скрытого состояния, который проходит через линейный слой для преобразования в логиты – ненормированные оценки, чтобы генерировать их в выходные токены и поступает в Softmax [11], чтобы преобразовать логиты в вероятностное распределение на словарь. Алгебраический вид, следующий:

$$P(w_i) = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}} \quad (1.13)$$

где z_i – логит для словаря w_i ,

V – размер словаря.

Особенность Softmax заключается в температуре, которая управляет гладкостью и резкостью распределения вероятностей. Значение температуры принимает два диапазона значений:

- От нуля до единицы. Разница между вероятностями становится великой, что подходит для более точных ответов.
- Более единицы. Разница между вероятностями становится минимальной.

Несмотря на выдающиеся достижения трансформеров в глубоком обучении, их применение ограничено из-за квадратичной сложности вычислений, не позволяющей эффективно обрабатывать длинные последовательности свыше 10 000 токенов, что существенно сужает их применение для обработки масштабных данных.

На основе данной модели была разработана Mamba [12]. В отличие от трансформеров, Mamba использует принципиально иную архитектуру – SSM (State Space Models) [13]. Хотя SSM появились гораздо раньше, в глубоком обучении они долгое время не могли сравниться по эффективности с трансформерами, особенно в задачах языкового моделирования. Однако Mamba преодолевает это ограничение: её вычислительная сложность линейна, а пропускная способность в 5 раз выше, чем у классических трансформеров.

SSM применяется для характеристики скрытых состояний системы и прогнозирования их возможных изменений в ответ на входные воздействия. С математической точки зрения, она реализуется через систему из двух ключевых уравнений: уравнения состояния, описывающего эволюцию системы, и уравнения наблюдения, определяющего выходные величины. Алгебраический вид, следующий:

$$h'(t) = Ah(t) + Bx(t), \quad (1.14)$$

$$y(t) = Ch(t) + Dx(t) \quad (1.15)$$

где $h'(t)$ – производная состояния;

A – матрица перехода состояний;

B – матрица входа;

$x(t)$ – входной сигнал;

$h(t)$ – скрытое состояние;

$y(t)$ – выходное состояние;

C – матрица наблюдения;

D – матрица сквозной связи.

Схемы работы SSM представлены на рисунках 1.10 – 1.13.

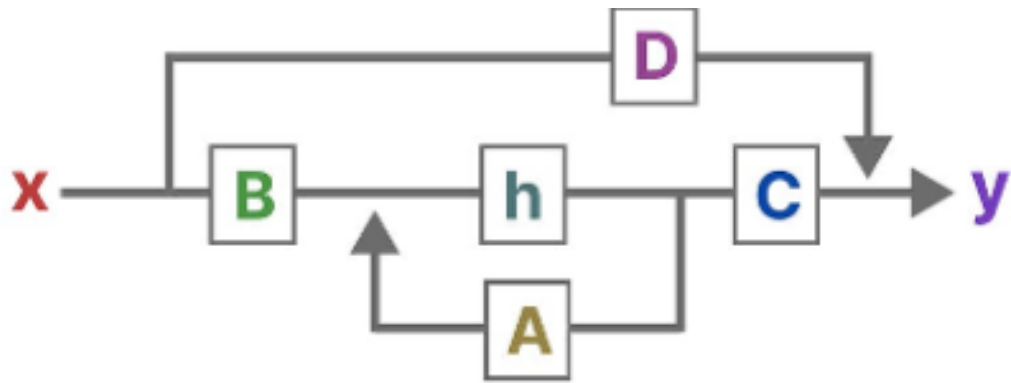


Рисунок 1.10 – Схема работы SSM

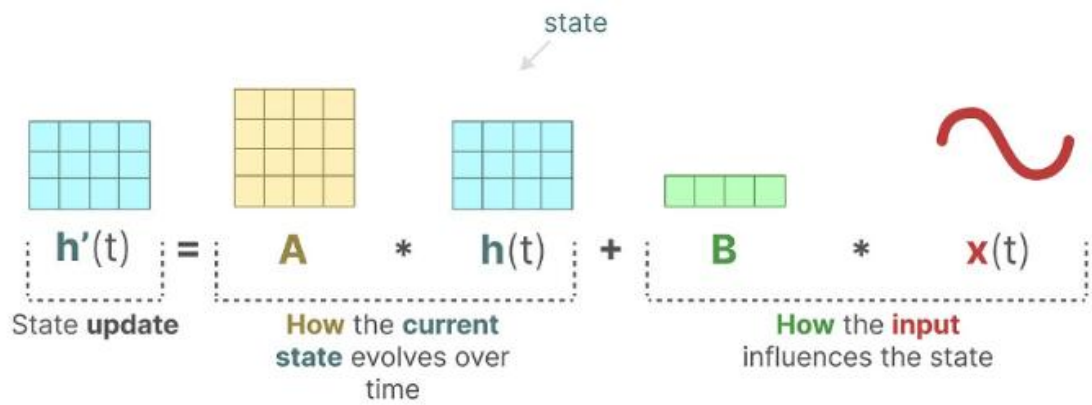


Рисунок 1.11 – Визуализация уравнение состояния

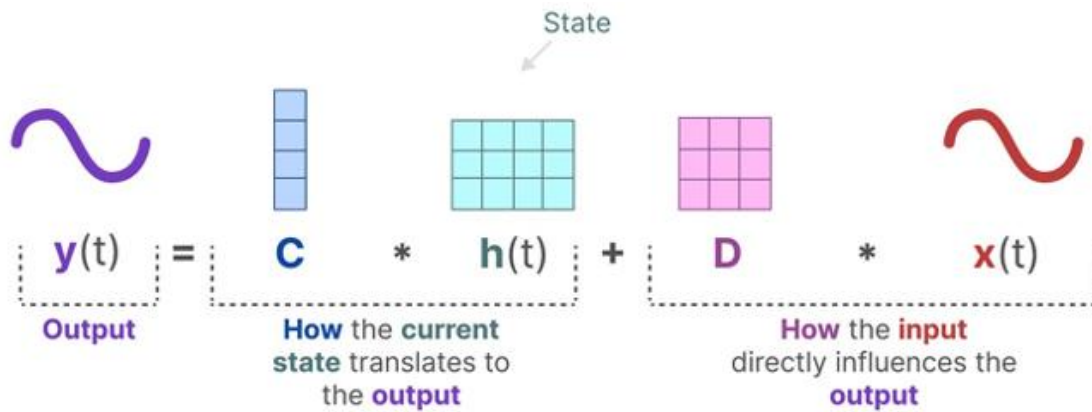


Рисунок 1.12 – Визуализация уравнение выхода

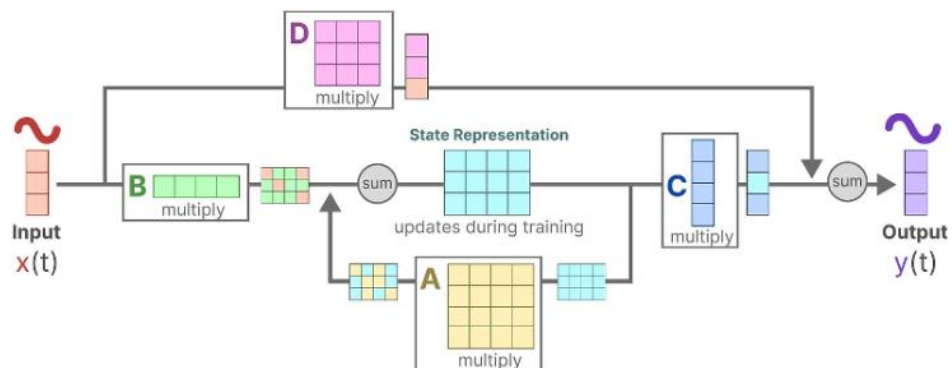


Рисунок 1.13 – Итоговая схема взаимодействия

Архитектура Mamba не отличается от трансформера, это можно увидеть на рисунке 1.14.

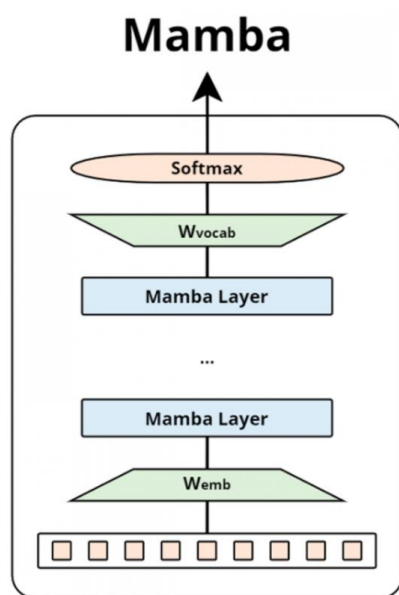


Рисунок 1.14 – Архитектура Mamba

Но по своему внутреннему строению слой Mamba отличается (рисунок 1.15).

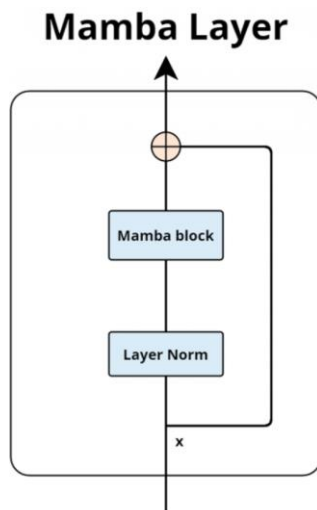


Рисунок 1.15 – Слой Mamba

Отличительной чертой является Mamba блок, в котором содержит SSM, как показано на рисунке 1.16.

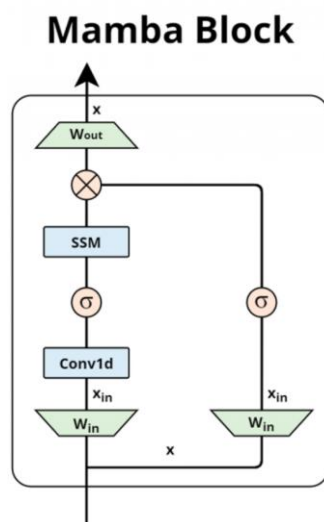


Рисунок 1.16 – Mamba блок

Блок использует модифицированную MLP-архитектуру с механизмом управления информационным потоком. Дополнительная ветвь, содержащая линейный слой и функцию активации, генерирует маску для поэлементного умножения, что позволяет избирательно усиливать или подавлять компоненты основного потока данных. Эта архитектура дополнена механизмом selective SSM, описанным ранее.

1.2.4 Выборка необходимых документов и данных

На данный момент, большинство общедоступных LLM при создании промпта ищут информацию в открытых интернет-источниках, но при поставленной цели и задачи необходимо искать среди тех данных, которые были предоставлены модели. Такой метод называется Retrieval Augmented Generation (RAG) [14] – это метод работы с большими языковыми моделями, когда пользователь пишет свой промпт, а система программно к этому вопросу «подмешивает» дополнительную информацию из каких-то внешних источников и подаете все целиком на вход языковой модели. Другими словами, добавление в контекст запроса к языковой модели дополнительную информацию, на основе которой языковая модель может дать пользователю более полный и точный ответ.

В текущем случае выборка будет происходить полностью из тех документов, которые находятся в базе данных корпорации.

На рисунке 1.17 отображена концепция RAG.

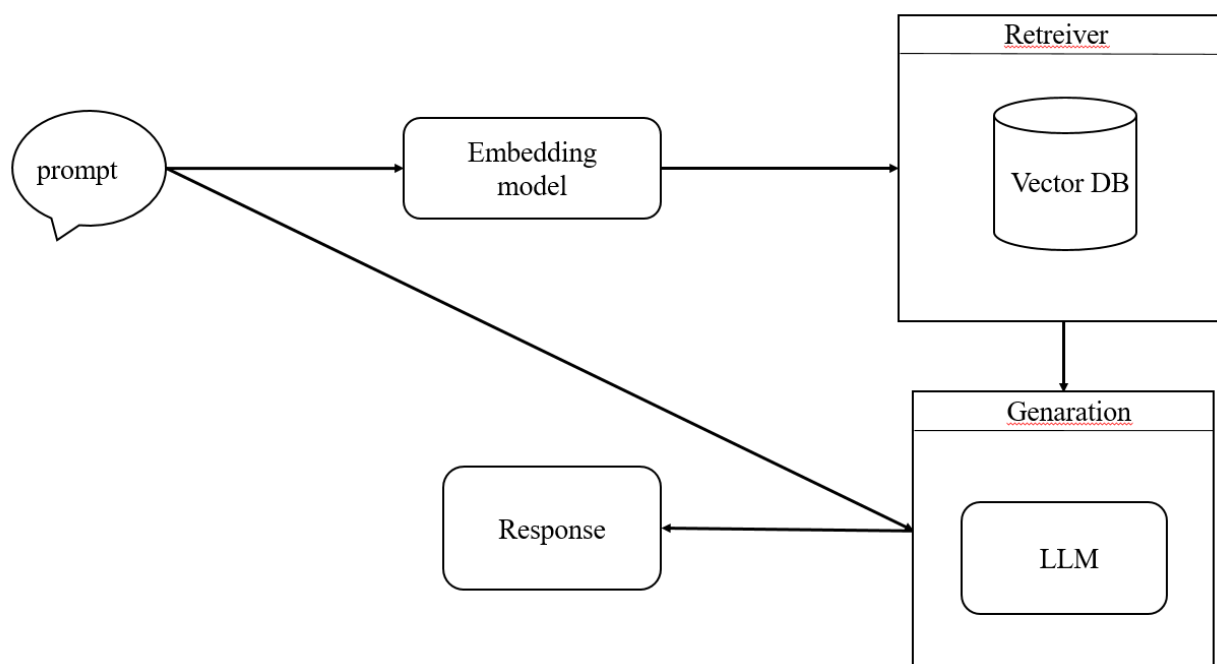


Рисунок 1.17 – Концепция RAG

Простыми словами, сотрудник вводит запрос, система его обрабатывает, ищет в корпоративных документах соответствующий документ, извлекает необходимую информацию, генерирующей через языковую модель, и в конце возвращает ответ на поставленный запрос.

При внедрении данного метода могут возникнуть следующие проблемы:

1. Нечеткий поиск – просто взять запрос и найти по точному соответствию все куски из документа не получится.
2. Размер данных документа – неизвестен размер «кусков» текста, который обрабатывает LLM.
3. На заданный запрос может найти несколько релевантных документов.

Размер текста в контексте больших языковых моделей называется чанками – небольшие куски текста, которые между собой объединяются и получается осмысленный ответ. Тогда выделить следующие идею:

1. Чем меньше чанк по размеру, тем точнее будет буквальный поиск, чем больше размер чанка тем больше поиск приближается к смысловому.

2. Разные запросы пользователя могут содержать разное количество чанков, которое необходимо добавлять в контекст. Необходимо опытным путем подобрать тот самый коэффициент, ниже которого чанк смысла не имеет и будет лишь замусоривать ваш контекст.

3. Чанки должны перекрывать друг друга, чтобы был шанс подать на вход последовательность чанков, которые следуют друг за другом вместе, а не просто вырванные из контекста куски.

4. Начало и конец чанка должны быть осмысленными, в идеале должны совпадать с началом и концом предложения, а лучше абзаца, чтобы вся мысль была в чанке целиком.

К определению более релевантного документа можно воспользоваться методами из раздела 1.2.2:

1. TF-IDF (вычисляет важность каждого слова в документе относительно количества его употреблений в данном документе и во всей коллекции текстов).

2. BM25 (функция ранжирования, используемая поисковыми системами для упорядочивания документов по их релевантности данному поисковому запросу).

Если же документы, лежащие в базе знаний, не упорядочены между собой, то можно воспользоваться графом знаний. Граф знаний – это направленный реляционный граф, где вершины обозначают сущности, а рёбра отражают связи между ними. Такие графы строятся на основе триплетов вида (*субъект, отношение, объект*) или (s, r, o) , что позволяет систематизировать информацию как о конкретных объектах, так и об абстрактных концепциях. Таким образом, граф знаний служит структурированной моделью для представления разнообразных фактов.

Таким образом документы можно структурировать между собой, обладая общей тематикой, что уменьшит время поиска и повысит точность. Пример такого графа представлен на рисунке 1.18.

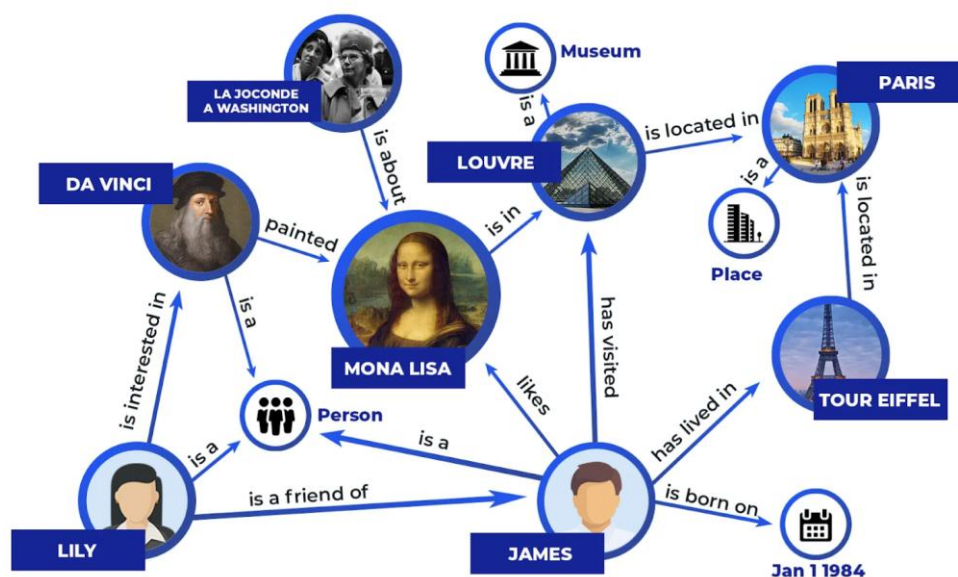


Рисунок 1.18 – Граф знаний

1.3 Развитие AI-агентов

Как было описано ранее, языковые модели автоматизируют различные процессы в компаниях, помогают пользователю в чем-то разобраться и т.п. Но что, если задач становится больше, чем одна, то одного запроса для решения может не хватить или же модель попросту забудет о этой задаче. Для этого были созданы AI-агенты. В свою очередь они выделяют в одной задаче несколько подзадач и решают каждую по отдельности. Так же, у них есть понимания, что у пользователя могут быть дополнительные данные, они собирают их и применяют для решения. В итоге, агенты – некий маленький офис, в котором трудятся офисные работники. На рисунках 1.19 – 1.20 представлены примеры расширений для агентов с точки зрения визуализации.



Рисунок 1.19 – Пример визуализации агентов



Рисунок 1.20 – Пример визуализации агентов

Тем самым, не сложно догадаться, что каждый агент выполняет некоторую роль в той или иной задаче. Если вам нужно, например, находить материалы в истории 20 века, то агент будет являться историком, который специализируется на 20 веке.

2. РЕАЛИЗАЦИЯ

В качестве примера будет представлен Telegram-Bot. В качестве языковой модели будет использоваться GigaChat от Сбербанка.

2.1 Получение необходимых параметров модели

Чтобы использовать GigaChat из вашего приложения, потребуется GigaChat API – интерфейс для интеграции с GigaChat. Для его настройки нам понадобятся несколько параметров, которые можно получить в проекте GigaChat API. Для этого перейдите в личное пространство в Developers Studio. Studio – это личный кабинет, где вы можете самостоятельно создать чат-бота. Зарегистрироваться в Studio можно по Сбер ID, СберБизнес ID или по email. Ссылка для входа: <https://developers.sber.ru/studio/login>. Поле входа представлено на рисунке 2.1.

The image shows a registration form titled "Регистрация" (Registration). It includes a link "Уже есть аккаунт? Войти" (Already have an account? Log in). The form has two input fields: "Почта" (Email) and "Пароль" (Password). The "Почта" field is empty and has a red error message "Отсутствует почта" (Email is missing). The "Пароль" field contains the placeholder text "Введите пароль" (Enter password) and has a red error message "Не менее 8 символов" (At least 8 characters). Below the password field is a checkbox with the text "Даю Согласие на получение рекламных сообщений и маркетинговых предложений" (I agree to receive advertising messages and marketing offers). At the bottom, there are three buttons: "Зарегистрироваться" (Register), "или" (or), and "Войти по номеру телефона" (Log in by phone number). There is also a button for "СберБизнес ID" (SberBusiness ID).

Регистрация
Уже есть аккаунт? [Войти](#)

Почта

Отсутствует почта

Пароль

Введите пароль

Не менее 8 символов

☐ Даю Согласие на получение рекламных сообщений и маркетинговых предложений

Зарегистрироваться

или

Войти по номеру телефона

СберБизнес ID

Рисунок 2.1 – Регистрация Studio

В личном пространстве необходимо создать проект с GigaChat API, по кнопке GigaChat API в разделе «Создать проект». На рисунке 2.2 показан порядок действий.

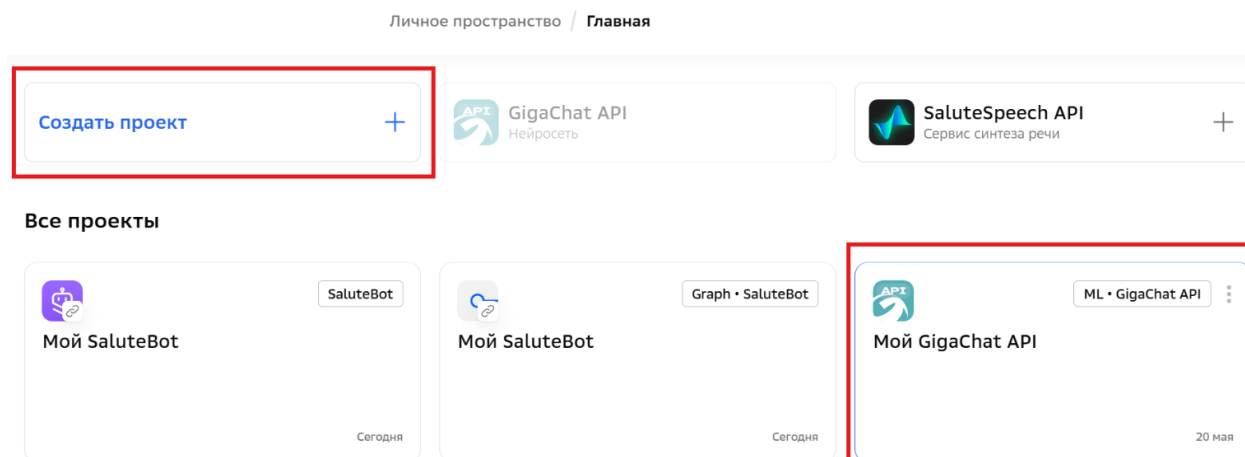


Рисунок 2.2 – Создание проекта

Затем выбираем созданный Мой GigaChat API — откроется раздел с настройками, как показано на рисунке 2.3.

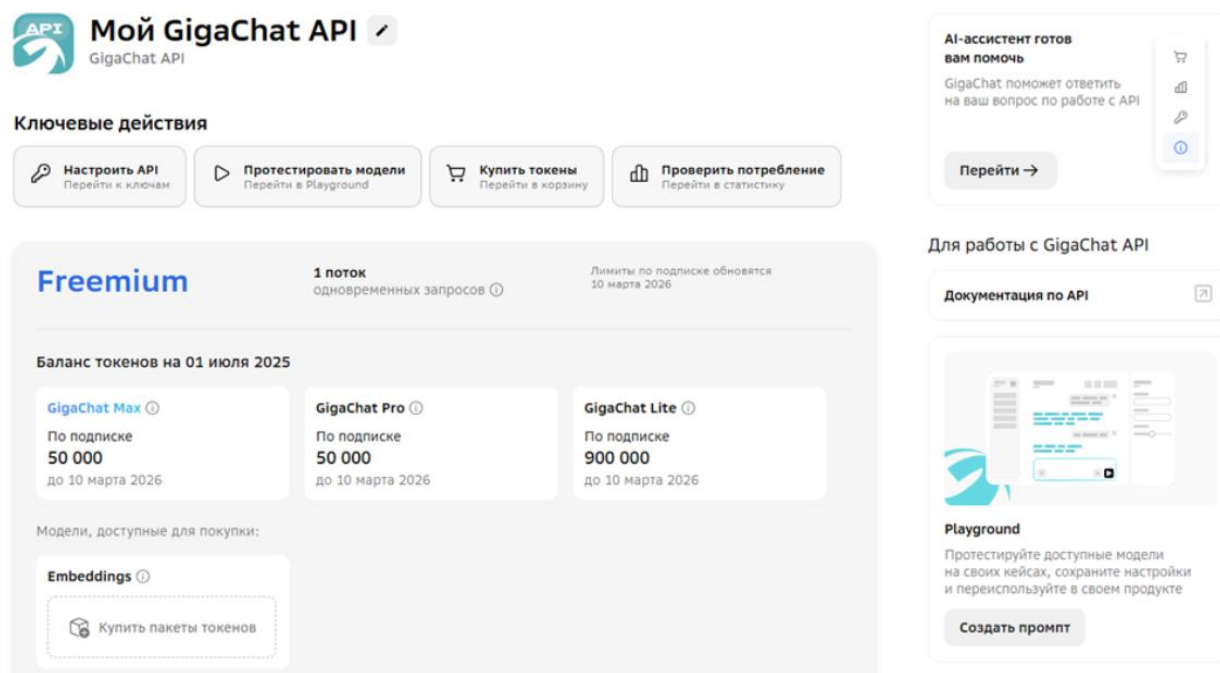


Рисунок 2.3 – Настройки проекта

В ключевых действиях выбираем "Настроить API" (рисунок 2.4).



Мой GigaChat API

Ключевые действия

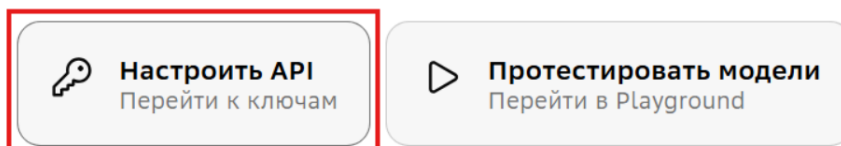


Рисунок 2.4 – Настройка API ключа

В этом разделе видим свои данные для авторизации, как показано на рисунке 2.5.

Настройка API

В этом разделе находятся данные для интеграции GigaChat в ваш продукт через API.

[Подробная документация о настройке API](#)

Данные для авторизации запросов к API

Client ID ⓘ
7cb22... 554b-4... bd53... eb

Scope ⓘ
GIGACHAT_API_PERS

Ключ авторизации

Передается в запросе на получение Access token

Authorization key
Вы ещё не запрашивали ключ

Получить ключ

Рисунок 2.5 – Данные авторизации

Нужно выпустить еще и Authorization Key – нажимаем "Получить ключ" (рисунок 2.6).

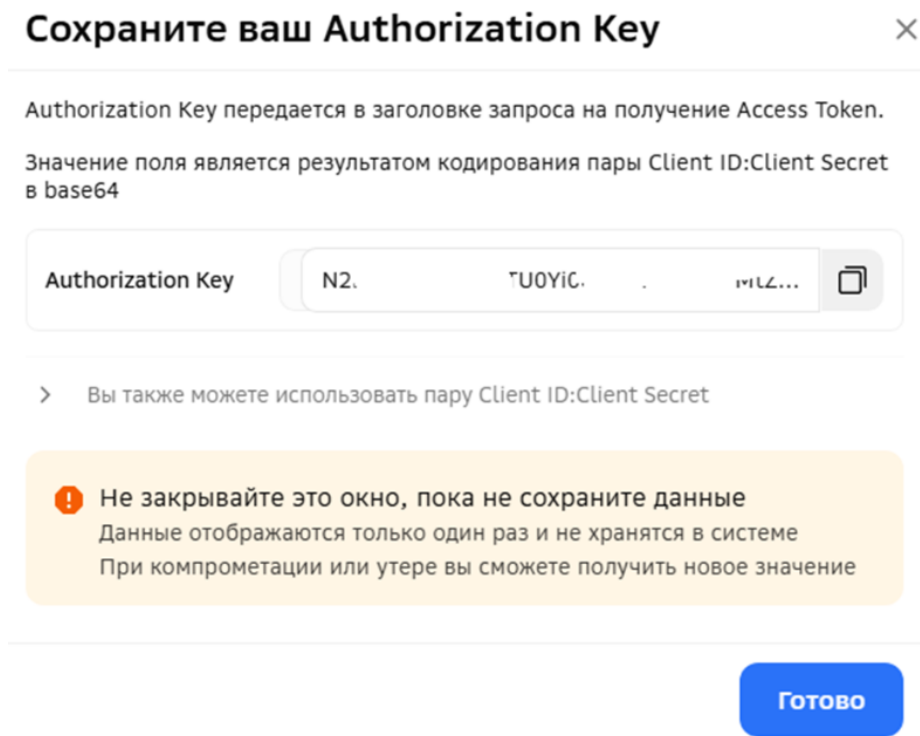


Рисунок 2.6 – Получение ключа авторизации

Вся дальнейшая реализация будет представлена в GitHub репозитории []. На рисунках 2.7 – 2.9 будет представлены ответы модели при разных запросах и конфигурациях.

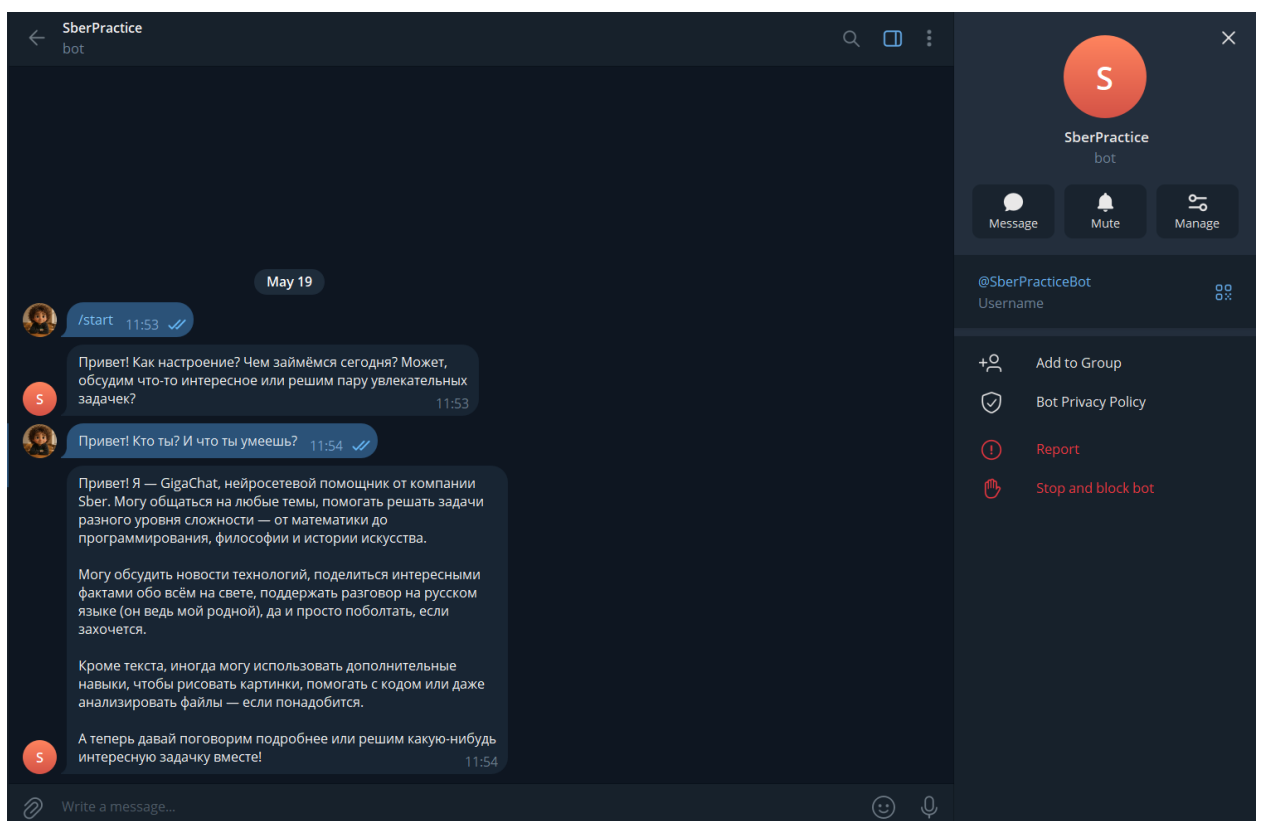


Рисунок 2.7 – Проверка инициализации и работоспособности бота

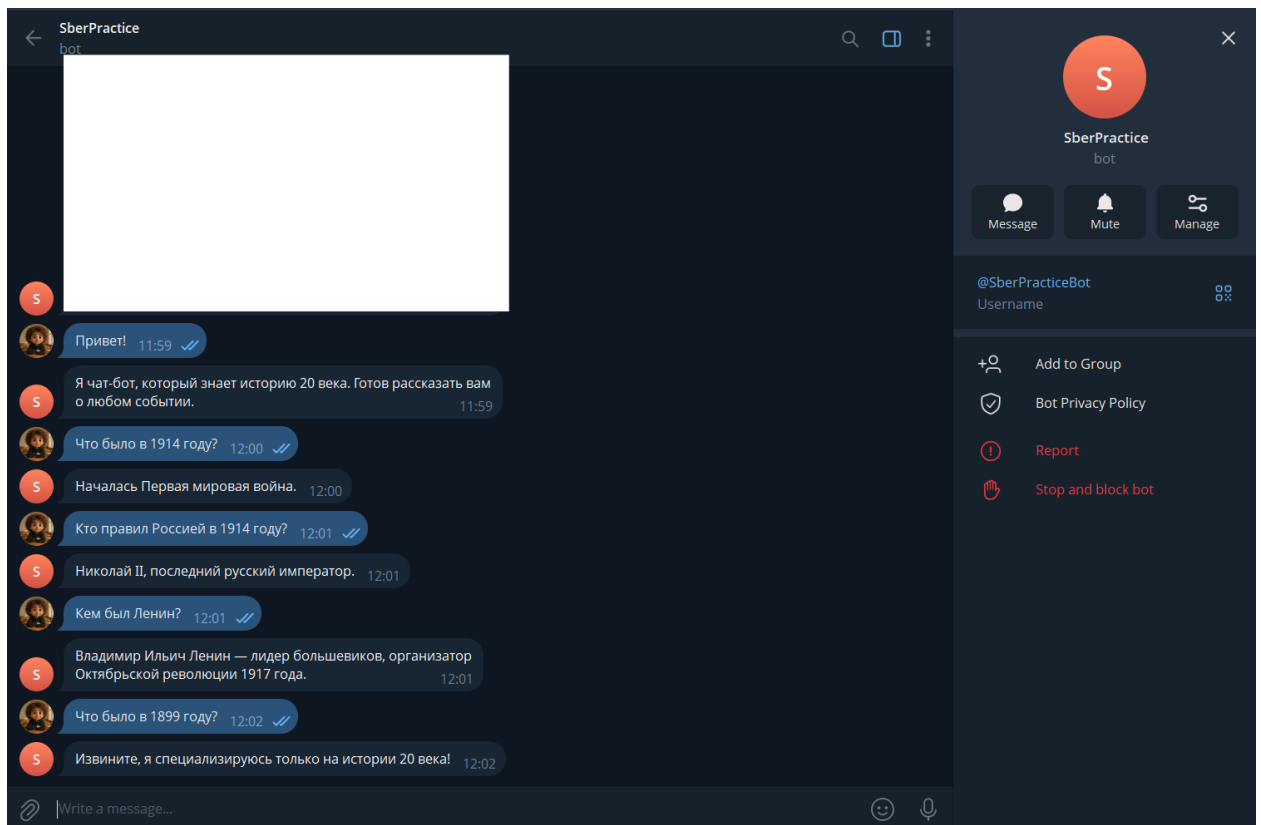


Рисунок 2.8 – Роль историка 20 века

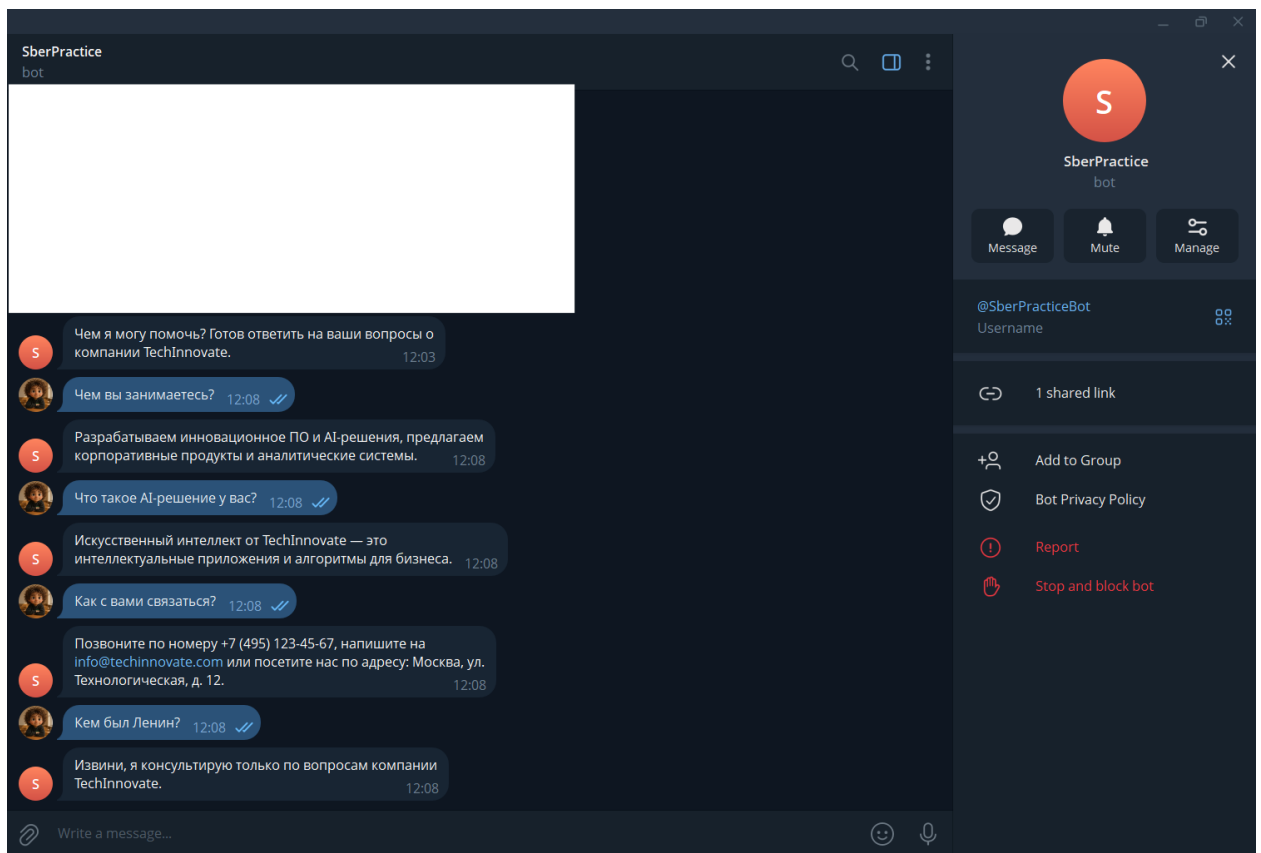


Рисунок 2.8 – Роль AI-компании

Заключение

В ходе прохождения практики были рассмотрены AI-агенты, их архитектура и отличия от классических LLM. Установлено, что агенты способны планировать действия, привлекать внешние инструменты и фильтровать нерелевантную информацию без анализа лишнего материала.

В теоретической части выполнен обзор эволюции NLP – от ранних систем до трансформеров и Mamba, а также методов векторизации, N-грамм и механизма внимания. Показана целесообразность RAG для работы с корпоративными данными и графов знаний для структурирования источников.

В практической части реализован Telegram-бот на GigaChat API, демонстрирующий работу AI-агента в ролевых задачах.

Результаты подтверждают, что AI-агенты сокращают время анализа документов, избегают информационного шума и повышают качество решений. Дальнейшее развитие связано с увеличением числа агентов, улучшением памяти и интеграцией с базами знаний.

Литературы

1. Christopher D. Manning Human Language Understanding & Reasoning [Электронный ресурс]: сайт American Academy of Arts and Sciences. URL: https://www.amacad.org/sites/default/files/publication/downloads/Daedalus_Sp22_09_Manning.pdf (дата обращения: 14.06.2026).
2. ChatGPT [Электронный ресурс]: сайт OpenAI. URL: <https://openai.com/stories/> (дата обращения: 14.06.2026).
3. Алан М. Тьюринг МОЖЕТ ЛИ МАШИНА МЫСЛИТЬ? [Электронный ресурс]: сайт Etheroneph. URL: https://www.etheroneph.com/files/can_the_machine_think.pdf (дата обращения: 14.06.2026).
4. Jamell Ivor Samuels One-Hot Encoding and Two-Hot Encoding: AnIntroduction [Электронный ресурс]: сайт ResearchGate. URL: https://www.researchgate.net/publication/377159812_One-Hot-Encoding_and_Two-Hot-Encoding_An_Introduction (дата обращения: 14.06.2026).
5. Neil Christian R. Riego, Danny Bell Villarba Utilization of Multinomial Naive Bayes Algorithm and Term Frequency Inverse Document Frequency [Электронный ресурс]: сайт arXiv. URL: <https://arxiv.org/pdf/2306.00018> (дата обращения: 14.06.2026).
6. Introduction to Information Retrieval [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/class/cs276/handouts/lecture12-bm25etc.pdf> (дата обращения: 14.06.2026).
7. N-gram Language Models [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/~jurafsky/slp3/3.pdf> (дата обращения: 14.06.2026).
8. The Transformer [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/~jurafsky/slp3/9.pdf> (дата обращения: 14.06.2026).
9. Introduction and Word2Vec [Электронный ресурс]: сайт Stanford. URL: https://web.stanford.edu/class/cs224n/readings/cs224n_winter2023_lecture1_notes_draft.pdf (дата обращения: 14.06.2026).

10. Vector Semantics and Embeddings [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/~jurafsky/slp3/6.pdf> (дата обращения: 14.06.2026).

11. Michael Franke, Judith Degen The softmax function: Properties, motivation, and interpretation [Электронный ресурс]: сайт Stanford. URL: https://alpslab.stanford.edu/papers/FrankeDegen_submitted.pdf (дата обращения: 14.06.2026).

12. Wrestling Mamba: Exploring Early Fine-Tuning Dynamics on Mamba and Transformer Architectures [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/final-projects/DanielGuoLucasEmmanuelBrennanAlmaraz.pdf> (дата обращения: 14.06.2026).

13. Hardware-aware Algorithms for Sequence Modeling [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/slides/cs224n-2024-lecture18-deployment-and-efficiency.pdf> (дата обращения: 14.06.2026).

14. AI Can Look Up StackOverflow too: Retrieval-Augmented Code Generation [Электронный ресурс]: сайт Stanford. URL: <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/slides/cs224n-2024-lecture18-deployment-and-efficiency.pdf> (дата обращения: 14.06.2026).